

Cybernetic Sabotage

Instructor Guide

Cybernetic Sabotage is a gamified SQL practice tool. Students take on the role of an investigator, writing real SQL queries against a live database to work through a 12-query case, with adaptive hints and an in-game handler guiding them along the way. This guide gives instructors everything needed to drop the activity into a class session with no additional prep.

Learning Objectives

By the end of this activity, students will be able to:

- Write `SELECT` queries with filtering, sorting, and row limits
- Use `JOIN` (including `LEFT JOIN`) to combine data across related tables
- Aggregate data with `GROUP BY`, `COUNT`, and `HAVING`
- Write correlated and nested subqueries
- Use `UPDATE`, `INSERT INTO`, `CREATE TABLE`, and `CREATE VIEW` (basic DDL/DML alongside querying)

Suggested Format

Time: A full playthrough (all 12 queries) typically takes 50–75 minutes. For a shorter session, Queries 1–7 (`SELECT`, `JOIN`, `GROUP BY`, `UPDATE`) fit comfortably in a 50-minute period; Queries 8–12 (views, subqueries, DDL/DML) work well as a second session or homework extension.

Setting: Works as an in-class lab (students on laptops, instructor circulating) or as a structured homework assignment with the debrief questions (below) discussed at the start of the next class.

Grouping: Works individually or in pairs. Pairs tend to work well for Queries 8–12, where the adaptive hints alone may not be enough for students new to subqueries.

The 12 Queries

#	Scenario	Concept(s)	Level
1	Retrieve every reported incident (all rows, all columns).	<code>SELECT *</code>	Beg.
2	Retrieve the single most recent incident.	<code>ORDER BY</code> , <code>LIMIT</code>	Beg.
3	Count incidents per robot model, including models with zero incidents.	<code>LEFT JOIN</code> , <code>GROUP BY</code> , <code>COUNT</code>	Int.
4	Count how many robots were updated in the past week.	<code>WHERE</code> (date range), <code>COUNT(DISTINCT)</code>	Int.
5	Identify employees who updated any robot in the past week.	Subquery, <code>IN</code>	Int./Adv.
6	Mark recently-serviced robots as “Under Repair,” then confirm the update.	<code>UPDATE</code> , <code>WHERE</code>	Int.
7	Find the employee with the most robot updates.	<code>GROUP BY</code> , <code>ORDER BY</code> , <code>LIMIT</code>	Int.

#	Scenario	Concept(s)	Level
8	Build a reusable view joining robots, incidents, and the employees who updated them.	CREATE VIEW, multi-table JOIN	Adv.
9	Find robot models involved in more than 2 incidents.	Subquery, GROUP BY, HAVING	Adv.
10	Design and create a new Repair table.	CREATE TABLE (DDL)	Beg.
11	Insert a new repair record into the table just created.	INSERT INTO	Beg.
12	Identify the most recent employee to update a robot currently under repair.	Subquery, MAX(), multi-table JOIN	Adv.

Each query includes three tiers of adaptive hints in-game (conceptual translation → query shape → near-complete scaffold), so students who get stuck are guided progressively rather than shown the answer outright.

Debrief Questions

Use these after the activity, either as a short class discussion or a written reflection:

1. Query 3 uses a LEFT JOIN rather than a plain JOIN. What would change in the results if you used a regular (INNER) JOIN instead, and why does that matter here?
2. Several queries (5, 9, 12) use a subquery instead of a single flat query. Could Query 9 be rewritten without a subquery? What would that look like?
3. What's the difference between filtering with WHERE and filtering with HAVING? Which queries in this set needed HAVING, and why couldn't WHERE do the same job there?
4. Which query took you the most attempts? Looking back, what was the actual misunderstanding, not just the syntax error, that was slowing you down?

Getting Access

This tool is already deployed and in active use; there is nothing to build or install. To bring it into your course, request access at:

cybernetic-sabotage-landing.vercel.app

Maintained by Hazra Imran, Northeastern University, Khoury College of Computer Sciences.